



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - Performance Measure
E - Environment
A - Actuators
S - Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width = width
self.height = height
self.walls = set(custom_walls)
self.food_positions = set()
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent
Because of the init method has the parameter `num_opponents=2` and `self.opponents` list is populated with opponent positions. Also, the agent must consider opponent movements when making decisions

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of all perceptions an agent has received over time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors component.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable

what the agent can see - agent_pos - Its own position, opponent_positions - all opponent positions, smells_food - Whether it's currently on food, score - Current score

what the agent cannot see - Complete list of all wall positions, Complete list of food positions, Future opponent movements, full state of the grid

So, the environment is partially observable because the agent does not have complete information about all aspects of the environment state. It cannot see walls or

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure component

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important to measure external environment changes rather than internal processing effort because the performance measure should evaluate actual outcomes, not how hard the agent is trying. If we measured internal effort, agents could exploit the system by simulating work without achieving real goals, as seen in the Vacuum World Exploit where an agent could just move around without cleaning and still score points. External outcomes are objective and verifiable, while internal effort is subjective and can be faked. The performance measure must evaluate what

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If we add toxic traps but hide them from the agent's sensors, the environment becomes more partially observable. The agent already couldn't see everything, and now there's another hidden element - the traps. This means the agent has even less information about the environment and must guess where traps might be, explore carefully, and remember where it found traps before.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The `smells_toxin` sensor helps the agent act rationally by telling it when it's standing on a dangerous trap. With this information, the agent can decide to move away and avoid losing 15 points. Without this sensor, the agent would step on traps by

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

Recall the vacuum world exploit...



Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING